武漢大學
WUHAN UNIVERSITY

《分布式软件原理与技术》
第七讲 服务治理（1）

计算机学院 贾向阳

武漢大學
WUHAN UNIVERSITY

1、 什么是服务治理?

微服务架构的可用性

高可用挑战

- 服务实例动态变化：服务重启、宕机、扩缩节点，造成调用方找不到服务地址
- 网络不可靠：超时、丢包、分区等造成请求无法到达或响应丢失
- 单点故障：某个服务实例崩溃，导致部分请求失败
- 故障传播（雪崩）：一个下游服务（如数据库）变慢，上游线程阻塞堆积，耗尽资源
- 突发流量冲击：请求超过系统容量，服务过载甚至宕机
- 配置错误：导致启动失败或运行异常，需重启修复，增加停机时间

服务治理

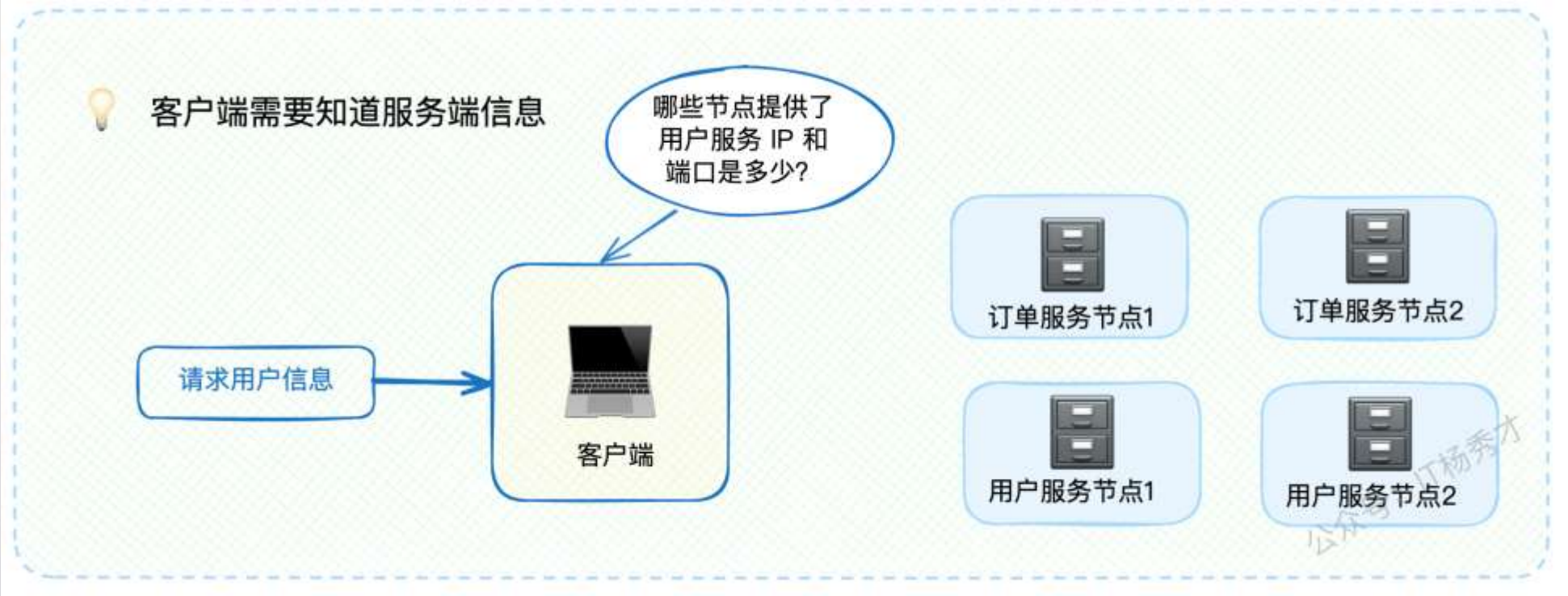
服务治理

- 服务治理是指对微服务架构中大量分散运行的服务进行协调、管理、控制，以保证整个系统高可用、可观测、运维可控的一系列机制与规范。

治理类别	治理目标	治理手段
高可用	保证系统持续存活，故障自愈	服务发现、负载均衡、熔断、限流、降级、超时控制
可观测	快速定位故障，理解系统行为	链路追踪、指标监控、日志聚合、健康检查
运维可控	降低运维复杂度，提升效率	配置中心、服务网格、容器编排、CI/CD、灰度发布

2、 服务注册与发现

为什么需要服务注册与发现?

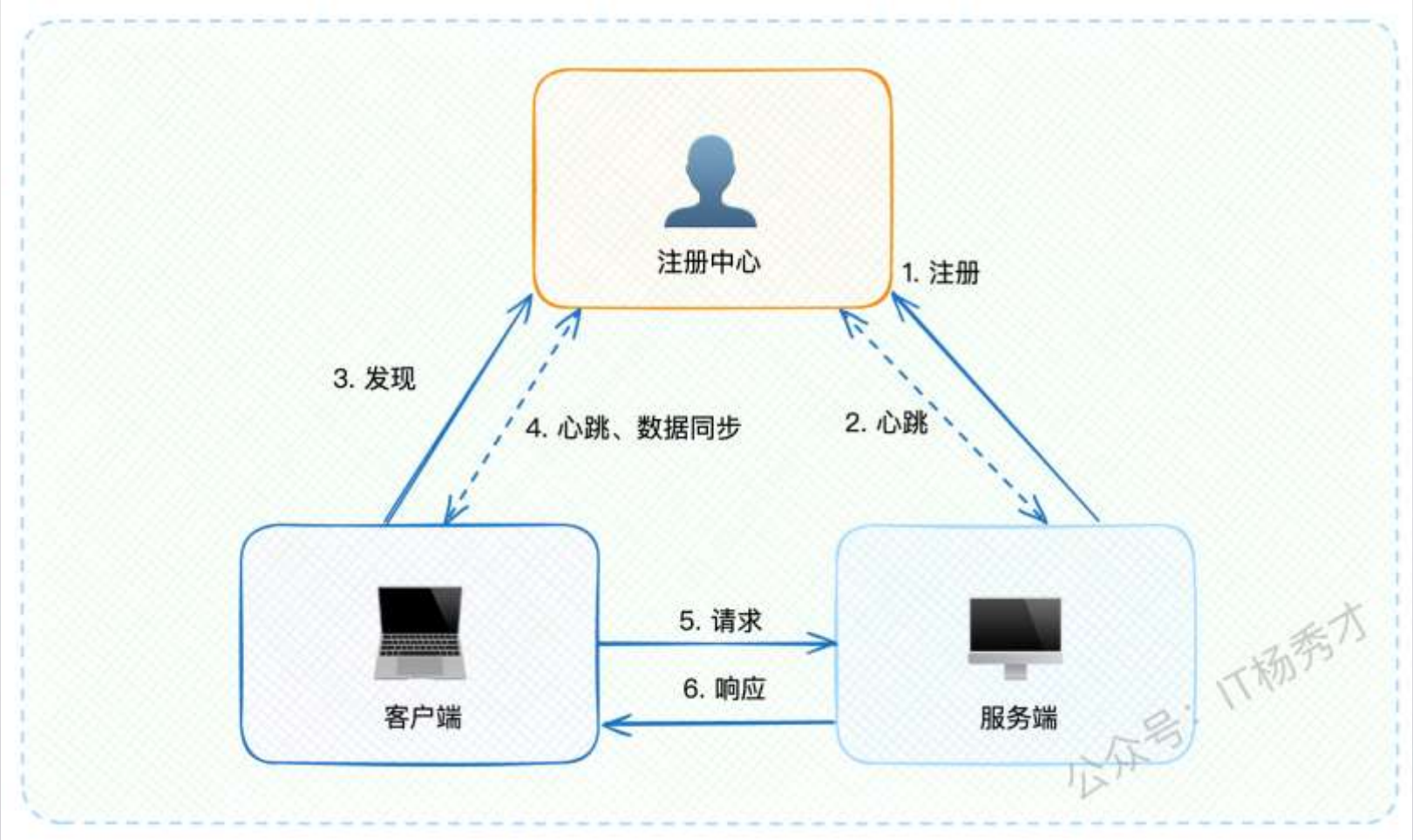


- 不使用服务注册时，调用方硬编码地址，或通过Nginx负载均衡器配置静态IP列表
- 如果服务器重启后地址变了，节点下线或者上线怎么办?

服务注册：用一个动态的服务注册表，让提供者注册自己，消费者查询可用节点。

武漢大學 WUHAN UNIVERSITY

服务注册与发现



服务注册的模式

（1）自注册模式

- 服务实例启动时主动向注册中心注册，关闭时主动注销；定期发送心跳。
- 典型实现：
  - Spring Cloud Eureka
  - Nacos
- 优点：实现简单，不需要额外组件
- 缺点：与语言/框架耦合；每个实例都要配置注册信息

服务注册的模式

（2）第三方注册

- 由外部代理（如K8s的kubelet、Consul的Registrator）自动检测服务启动并注册
- 典型实现：
  - Kubernetes：Pod启动后，kubelet将Pod节点的IP和Port注册到Endpoint
  - Consul + Registrator
  - 服务网格（Istio）：Envoy代理自动注册
- 优点：对业务代码无侵入；适合多语言环境
- 缺点：需要额外的进程或基础设施支持

Eureka服务注册

注册中心：在一个SpringBoot项目中添加配置，成为一个EurekaServer服务

```yaml
//Eureka Server的application.yml 配置
server:
  port: 8761
eureka:
  client:
    registerWithEureka: false
    fetchRegistry: false
```

```java
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.cl
    }
}
```

其他微服务实现为Eureka Client

```yaml
// application.yml 配置
eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka/
  instance:
    leaseRenewalIntervalInSeconds: 30   // 心跳间隔
    leaseExpirationDurationInSeconds: 90 // 失效时间
```

```java
// 主类上添加注解
@SpringBootApplication
@EnableEurekaClient   // 开启自注册
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}

// 服务关闭时自动注销（Spring Boot 优雅关闭自动发送取消请求）
```

Nacos服务注册

```yaml
# 项目的application.yaml文件
spring:
  application:
    name: springboot-nacos
nacos:
  discovery:
    server-addr: 127.0.0.1:8848
    #是否将本应用注册到nacos服务列表，默认是false
    auto-register: true
    #本服务所属的命名空间的ID，默认是空，也就是public
    namespace: c91a7f69-cefc-4ee3-8350-07ab87affd05
    register:
      #本服务是否接受外部的请求，默认true
      enabled: true
      #本服务要注册到命名空间下的哪个组，默认DEFAULT_GROUP
      group-name: DEFAULT_GROUP
      #服务的名称，会展示在nacos服务列表，要求唯一，可以不写，默认是spring.application.name的值
      service-name:
```

k8s中的容器部署与服务注册

```yaml
# backend-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-app
  labels:
    app: backend
spec:
  replicas: 2  # 部署了两个后端Pod（有不同的IP地址）
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: backend-container
        image: your-backend-image:1.0   # Replace with your actual backend image
        ports:
        - containerPort: 8080
```

- 应用中的多个容器作为一个整体，部署在节点上（称为Pod）。
- 一个应用可以部署多个Pod副本实现负载均衡

```yaml
# backend-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend   # This matches the labels on the backend Pods
  ports:
  - protocol: TCP
    port: 80        # Service port
    targetPort: 8080 # Pod's container port
  type: ClusterIP   # Default, internal to the cluster
```

- 注册服务，将应用分配一个稳定的ClusterIP，将内部容器端口映射出来。
- 通过ClusterIP可以调用容器内的SpringBoot程序，并实现负载均衡

服务发现的模式

（1）客户端发现

- 消费者从注册中心获取提供者列表，自行负载均衡和调用
- 典型实现：
  - Eureka + Ribbon
  - Nacos + Spring Cloud LoadBalancer
- 优点：去中心化，无单点瓶颈；协议灵活
- 缺点：每种语言/框架需实现发现逻辑；维护成本高

服务发现的模式

（2）服务端发现

- 消费者通过中间代理（负载均衡器）发起请求，代理负责查询注册中心和转发
- 典型实现：
  - Kubernetes Service
  - AWS ALB
  - Nginx + Consul Template
- 优点：对应用透明；多语言复用
- 缺点：代理成为潜在性能瓶颈和单点故障（可多副本）

Spring Cloud LoadBalancer访问Nacos服务

```yaml
server:
  port: 8080

spring:
  application:
    name: consumer-service
  cloud:
    nacos:
      discovery:
        server-addr: 127.0.0.1:8848
    loadbalancer:
      cache:
        enabled: true              # 启用服务列表缓存
        ttl: 35s                   # 缓存过期时间
      ribbon:
        enabled: false             # 确保使用 Spring Cloud LoadBalancer，而非 Ribbon
```

```java
@Configuration
public class LoadBalancerConfig {
    @Bean
    @LoadBalanced   // 关键注解：让 RestTemplate 拥有负载均衡能力
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
public class ConsumerController {
    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/test")
    public String testLoadBalancer(@RequestParam String msg) {
        // 使用服务名 "provider-service" 代替具体的 IP:Port
        String url = "http://provider-service/echo?message=" + msg;
        // RestTemplate 会自动从 Nacos 获取 provider-service 的实例列表，
        // 并通过 LoadBalancer 选择一个实例发起请求。
        return restTemplate.getForObject(url, String.class);
    }
}
```

K8S service服务发现与调用

```java
// OrderController.java
@RestController
public class OrderController {

    // 使用 K8s Service 的 DNS 名称（命名空间可省略，因为在同一 namespace 下）
    private static final String USER_SERVICE_URL = "http://user-service:8080/user/";

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/order/{userId}")
    public Map<String, Object> createOrder(@PathVariable Long userId) {
        // 调用 user-service，K8s 会自动负载均衡到后端 Pod
        String url = USER_SERVICE_URL + userId;
        Map<String, String> userInfo = restTemplate.getForObject(url, Map.class);

        Map<String, Object> order = new HashMap<>();
        order.put("orderId", System.currentTimeMillis());
        order.put("userId", userId);
        order.put("userInfo", userInfo);
        order.put("message", "Order created by order-service");
        return order;
    }
}
```

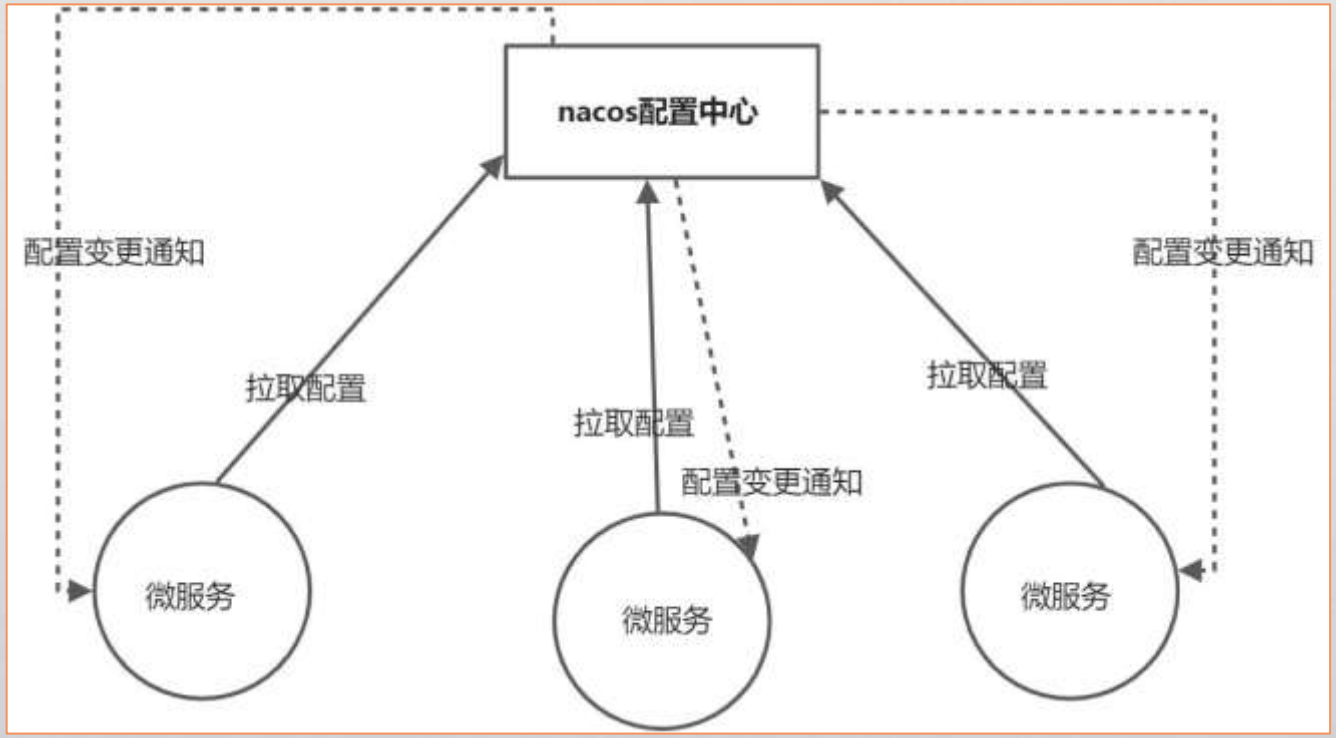
3、 配置管理

从本地配置到配置中心

- **传统做法**
  - 配置写在 application.properties 或 application.yml 中，打包进应用。
- **传统方法的问题**
  - 配置分散：每个服务有自己配置文件，修改一个参数需要改动几十个地方。
  - 环境差异：开发、测试、生产环境配置不同，管理混乱。
  - 修改需重启：任何配置变更必须重新打包、发布、重启服务 → 停机时间。
  - 安全风险：数据库密码、API密钥明文写在代码仓库中。

配置中心：集中管理、动态刷新、版本控制

功能	说明
集中存储	所有服务的配置文件存储在配置中心服务端，支持多格式（Properties、YAML、JSON）
环境隔离	通过 Namespace/Group 区分开发、测试、生产环境
动态刷新	客户端监听配置变更，收到通知后重新加载配置
版本管理	每次修改生成新版本，支持一键回滚
权限控制	配置修改需要认证，可设置读写权限
内容校验	支持格式校验（如 YAML 语法检查），避免错误配置上线



配置中心的技术选型

特性	Nacos	Apollo	Spring Cloud Config	K8s ConfigMap
动态刷新	✅ 支持（通过 @RefreshScope）	✅ 原生支持，实时推送	⚠️ 需结合 Spring Cloud Bus + MQ	❌ 需要重启 Pod 或手动触发滚动更新
配置格式	Properties、YAML、JSON、XML	Properties、YAML、XML	同 Spring Boot	键值对，可挂载为文件
版本管理	✅ 支持	✅ 支持，可视化回滚	❌ 依赖 Git 版本	✅ 可通过 kubectl rollout history
权限控制	✅ 支持 RBAC	✅ 完善的权限管理	❌ 无内置	✅ 依赖 K8s RBAC
多环境隔离	Namespace + Group	Environment + Cluster	Profile	Namespace
配置监听	长轮询（30s）	长连接（HTTP 轮询）	需要手动实现	无（需重启 Pod）
适用场景	Spring Cloud 生态，同时做服务发现	大规模、复杂配置管理的企业	简单 Spring Boot 项目	K8s 原生应用，配置不常变更

SpringBoot+Nacos配置管理

```java
@RestController
public class ConfigController {

  @Value("${plainKey}")
  String testKey;

  //* NacosConfig支持运行期动态更新属性值 */
  @NacosConfig(dataId = "routeconfig", group = "config", key = "rate")
  String rate;

  @RequestMapping("/testPlainKey")
  public String test() {
    return testKey;
  }

  @RequestMapping("/rate")
  public String rate() {
    return rate;
  }

}
```

K8s ConfigMap示例

- **ConfigMap**
  - K8s 对象，用于存储配置数据
- **使用方法**
  - 作为环境变量注入 Pod（建议）
  - 作为文件挂载到 Pod 的某个目录
- **动态更新行为**
  - 环境变量方式：修改 ConfigMap 后，不会更新已运行 Pod 的环境变量，必须重启 Pod。
  - 文件挂载方式：修改 ConfigMap 后，kubelet 会同步更新挂载文件，应用需要自己监控文件变化并重新加载配置。

```java
@RestController
public class ConfigController {

    @Value("${app.rate.limit:100}")
    private int rateLimit;

    @Value("${app.name:unknown}")
    private String appName;

    @GetMapping("/config")
    public Map<String, Object> getConfig() {
        return Map.of("rateLimit", rateLimit,
    }
}
```

```yaml
# configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  APP_RATE_LIMIT: "1000"
  APP_NAME: "order-service"
```

```yaml
# deployment.yaml 注入环境变量
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  spec:
    template:
      spec:
        containers:
        - name: my-container
          image: myapp:latest
          env: #设置容器使用的环境变量
          - name: APP_RATE_LIMIT
            valueFrom:
              configMapKeyRef:
                name: my-config
                key: APP_RATE_LIMIT
          - name: APP_NAME
            valueFrom:
              configMapKeyRef:
                name: my-config
                key: APP_NAME
```

4、 服务网关

为什么需要服务网关?

场景

- 假设我们已搭建好以下微服务系统：
  - 订单服务（Order Service）—— 3个实例，地址动态变化
  - 用户服务（User Service）—— 2个实例
  - 商品服务（Product Service）—— 4个实例
- 这些服务都已注册到 Nacos注册中心，服务之间可以互相发现和调用。
- **问题：**手机 App 或 Web 前端这些外部应用，如何调用这些服务?

早期的服务网关：反向代理

- 反向代理（如Nginx）采用手工静态配置服务地址。不能很好支持服务实例动态变化、API频繁迭代，以及灰度发布和AB测试等场景，不支持熔断限流降级等流量治理与容错。

现代服务网关的能力

- 现代服务网关（如Spring Cloud Gateway、Kong、APISIX）在反向代理基础上增加了以下能力：

功能	网关的实现方式
动态路由	根据请求URL、Header、参数、用户身份等精细化转发
服务发现	集成Nacos/Eureka/K8s等注册中心，动态感知实例变化
认证授权	原生支持JWT、OAuth2.0、API Key，可自定义认证逻辑
精细化限流	按接口、用户、角色、场景做限流，支持令牌桶、滑动窗口等算法
熔断降级	下游故障时快速失败，防止级联雪崩
协议转换	HTTP ↔ gRPC ↔ Dubbo 等多协议互转
请求聚合	调用多个后端服务，聚合结果一次返回
灰度发布	基于Header/Cookie/用户ID等条件，按比例或规则路由到不同版本
可观测性	内置日志、指标、链路追踪，可对接Prometheus、Jaeger等

服务网关的选型

特性	Spring Cloud Gateway	Kong	APISIX	Nginx（作为网关）
架构	基于 Spring WebFlux（响应式）	基于 Nginx + OpenResty（Lua）	基于 Nginx + etcd（Lua）	基于 Nginx（C）
动态路由	✅ 与注册中心集成（Nacos/Eureka）	✅ 通过 Kong Admin API 或 DB	✅ 通过 Admin API	❌ 需 reload 配置
插件生态	Java 编写，可自定义	Lua 插件丰富	Lua/Java/Python/Go 插件	需编译第三方模块
性能	较高（响应式）	高	极高	极高
语言偏好	Java 技术栈	多语言	多语言	任何语言
学习曲线	中等（需 Spring 知识）	中等（Lua）	中等	低（但动态配置难）
适用场景	Spring Cloud 微服务架构	云原生、混合技术栈	高性能、动态配置要求高	简单静态路由或作为k8s Ingress

Spring Cloud Gateway + Nacos的服务动态路由

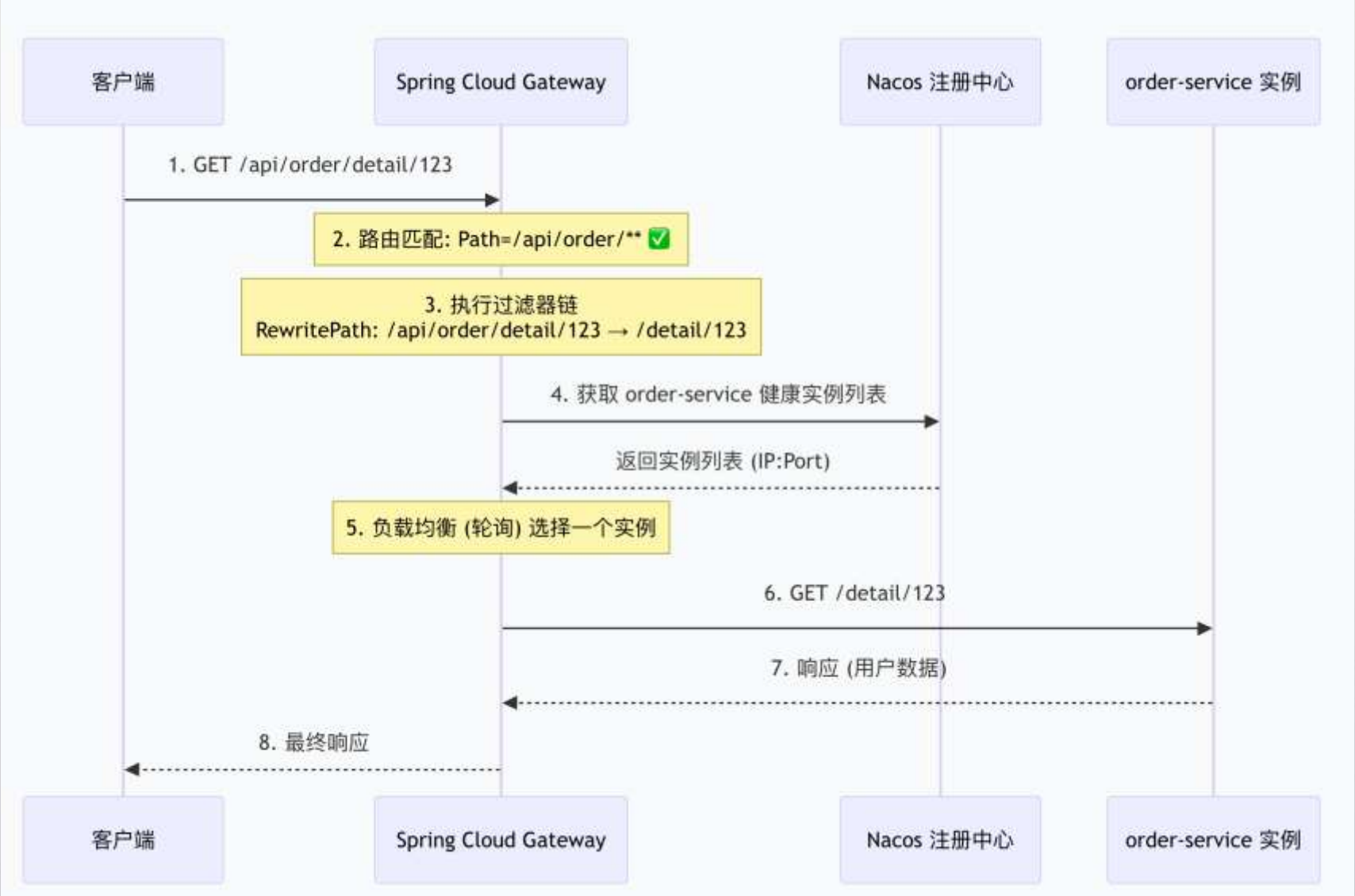
```yaml
# application.yml
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    nacos:
      discovery:
        server-addr: 127.0.0.1:8848
    gateway:
      routes:
        - id: order-service-route
          uri: lb://order-service          # lb:// + Nacos中的服务名
          predicates:
            - Path=/api/order/**
          filters:
            - RewritePath=/api/order/?(?<segment>.*), /$\{segment}
            #重写路径。如内部服务/api/order/detail/123 被映射为 http://网关IP:8080/api/order/detail/123

        - id: user-service-route
          uri: lb://user-service
          predicates:
            - Path=/api/user/**
          filters:
            - RewritePath=/api/user/?(?<segment>.*), /$\{segment}
```

使用Spring Cloud Gateway，需要自建一个SpringBoot项目并配置对应的路由和过滤规则。

Spring Cloud Gateway + Nacos的服务动态路由



武漢大學
WUHAN UNIVERSITY

K8s中的服务网关

- **K8s Ingress Controller**
  - K8S的第一代网关标准，定义一个‘入口规则’，告诉外部请求如何访问集群内的服务
- **Gateway API**
  - K8S的第二代网关标准，提供更丰富的协议支持、流量治理与权限认证能力

Features	Ingress Controller	Gateway API
Protocol support	Primarily HTTP	HTTP, gRPC, TCP, and potentially more in the future
Routing rules	Path and host-based	Path, host, and arbitrary header-based
Configuration	Static YAML files	Dynamic configuration through the Kubernetes API server
Extensibility	Limited, relies on annotations	Extensive and supports custom filters and policies
RBAC	Not natively supported	Supports RBAC for granular access control

ngrok

K8s中的服务网关

➢ K8S中的服务网关选型

网关	支持标准	特点	适用场景
Traefik	Ingress + Gateway API	动态配置，自动服务发现，事件驱动	动态环境、自动化的中小型团队
Kong	Ingress + Gateway API	插件生态丰富，企业级功能完备	多语言环境、需要丰富插件
Nginx Gateway Fabric	Gateway API	Nginx 官方推出的 Gateway API 实现，是 ingress-nginx 的直接替代	希望延续 Nginx 生态的用户
Envoy Gateway	Gateway API	基于 Envoy 代理，高性能，丰富的流量管理	高性能、企业级治理能力
Istio	Gateway API	服务网格的流量入口，支持 mTLS、全链路可观测	已使用/计划使用服务网格的团队
Higress	Ingress + Gateway API	阿里云推出的云原生统一流量网关、微服务网关和安全网关	国内企业、需要全生命周期 API 管理
APISIX	Ingress + Gateway API	高性能、云原生 API 网关	对性能、动态性要求极高的云原生环境

5、 流量治理

什么是流量治理?

- 定义
  - 流量治理是对微服务之间的请求流量进行路由、控制、调度和保护的一系列技术手段。
- 目标
  - 保证高可用（防止雪崩、过载）
  - 保证高可靠（请求正确完成）
  - 支持平滑发布（灰度发布、金丝雀发布）
  - 提升可观测性（流量镜像、采样）

流量治理的手段

类别	手段	解决的问题
路由控制	灰度发布、金丝雀、AB测试、按版本/地域/用户路由	平滑上线新版本，降低发布风险
弹性控制	超时、重试、熔断、限流、降级	防止故障扩散，保护系统稳定性
流量管理	流量镜像、流量复制、流量采样	测试新版本、监控分析
负载均衡	轮询、加权、最小连接数、一致性Hash	均衡分发，避免热点

路由控制 —— 灰度发布与金丝雀

- **灰度发布（Gray Release）**
  - 新版本逐步放量，从少数用户到全部用户。
- **金丝雀发布（Canary Release）**
  - 先让少量实例运行新版本，观察无问题后全量替换。
- **实现方式**
  - 基于权重的路由：例如 90% 流量到 v1，10% 到 v2。
  - 基于条件的路由：例如携带 x-version: v2 Header 的请求到 v2，其余到 v1；或按用户 ID 哈希。

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: order-v1
          uri: lb://order-service-v1
          predicates:
            - Path=/order/**
          filters:
            - name: Weight
              args:
                group: order-canary
                weight: 90 #路由权重为90
        - id: order-v2
          uri: lb://order-service-v2
          predicates:
            - Path=/order/**
          filters:
            - name: Weight
              args:
                group: order-canary
                weight: 10 #路由权重为90
```

弹性控制

- 弹性控制是保障高可用和高可靠的关键，让系统在故障中“优雅求生”

手段		作用	典型配置
超时	调用下游超过指定时间直接失败，释放线程资源。	避免线程阻塞	连接超时 1s，读取超时 3s
重试	对失败请求再次尝试，需保证幂等性	提高成功率	最多重试 2 次，指数退避
熔断	当下游服务错误率超标时，快速失败，不再发送请求，防止级联故障	防止雪崩	错误率 > 50% 时熔断，5s 后半开
限流	限制单位时间内的请求数量，保护系统不被冲垮	保护自身	每秒 1000 请求，令牌桶算法
降级	当依赖服务不可用时，返回兜底数据或执行备选逻辑。	有损服务	返回缓存数据或默认值

流量治理技术方案

方案	代表技术	实现位置	优点	缺点
SDK嵌入	Spring Cloud Hystrix, Resilience4j（官方推荐)	应用代码内	功能丰富，灵活定制	多语言重复实现，升级需改代码
网关层	Spring Cloud Gateway, Kong等	统一入口	集中治理，对外友好	无法处理服务间流量
K8s 原生	Ingress + Service + NetworkPolicy	平台层	无需额外组件，简单场景够用	治理能力弱，难以精细化
服务网格	Istio, Linkerd	Sidecar 代理	业务无侵入，多语言统一，可观测性强	复杂度高，性能有损耗

电商大促流量治理案例

- 场景
  - 电商平台计划在“618”大促期间开展秒杀活动，需要在活动期间进行流量治理保障服务可用。
- 系统架构
  - 入口：Spring Cloud Gateway（部署在 K8s 内）
  - 服务：订单服务、库存服务、用户服务、促销服务、推荐服务
  - 注册中心：Nacos
  - 配置中心：Nacos（动态调整限流阈值、熔断开关）

电商大促流量治理案例

治理目标

- 保证核心下单流程高可用（99.99%）
- 非核心服务（推荐）可降级
- 支持秒杀期间的精准限流

挑战	风险
突发流量冲击	订单服务被瞬时流量打垮，整个系统不可用
库存服务响应变慢	上游订单服务线程阻塞，引发雪崩
推荐服务依赖第三方	第三方接口不稳定，影响主流程

限流配置

- 两层限流配置

限流位置	目标	阈值	算法
网关（全局）	总入口保护	5000 QPS	令牌桶：允许突发流量，令牌以固定速率生成
订单服务（秒杀接口）	防止单一接口打爆	1000 QPS	滑动窗口：强制平滑流量，处理速率固定
库存服务	保护数据库	800 QPS	令牌桶

限流配置

Spring Cloud Gateway网关限流

- RequestRateLimiter：基于 Redis 的令牌桶算法
- key-resolver：限流的键（如 IP、用户 ID、API 路径）
- redis-rate-limiter.replenishRate：令牌填充速率（每秒新增令牌数）
- redis-rate-limiter.burstCapacity：令牌桶容量（允许的最大突发请求数）。

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: global-limit
          uri: lb://order-service
          predicates:
            - Path=/order/**
          filters:
            - name: RequestRateLimiter
              args:
                key-resolver: "#{@ipKeyResolver}"
                redis-rate-limiter.replenishRate: 1000
                redis-rate-limiter.burstCapacity: 1500
```

```java
@Bean
public KeyResolver ipKeyResolver() {
    return exchange -> Mono.just(
        exchange.getRequest().getRemoteAddress().getHostName()
    );
}
```

武漢大學 WUHAN UNIVERSITY

限流配置

- 服务层限流配置（Resilience4j）

```yaml
resilience4j:
  ratelimiter:
    instances:
      # 订单服务全局限流
      order-service-global:
        limitForPeriod: 2000        # 每个周期内允许的请求数
        limitRefreshPeriod: 1s      # 周期长度
        timeoutDuration: 0s         # 等待超时时间（0=直接拒绝
      # 秒杀接口单独限流
      seckill-api:
        limitForPeriod: 100
        limitRefreshPeriod: 1s
        timeoutDuration: 0s
      # 用户维度限流（动态解析）
      user-specific:
        limitForPeriod: 10
        limitRefreshPeriod: 1s
```

```java
@RestController
public class OrderController {

    @RateLimiter(name = "order-service-global")
    @GetMapping("/order/create")
    public String createOrder(@RequestParam String userId) {
        // 正常业务逻辑
        return "order created";
    }
    @RateLimiter(name = "seckill-api")
    @PostMapping("/seckill")
    public String seckill(@RequestParam String skuId) {
        // 秒杀逻辑
        return "seckill success";
    }
    // 动态限流键（如按用户ID）
    @RateLimiter(name = "user-specific", fallbackMethod = "rateLimitFallback")
    @GetMapping("/user/info")
    public String getUserInfo(@RequestParam String userId) {
        return "user info for " + userId;
    }

    public String rateLimitFallback(String userId, Throwable t) {
        return "请求过于频繁，请稍后再试";
    }
}
```

熔断与重试配置（Resilience4j）

```yaml
resilience4j:
  circuitbreaker:
    instances:
      inventory-cb:
        sliding-window-size: 20            # 统计窗口大小（请求次数）
        failure-rate-threshold: 50         # 失败率阈值（百分比）
        wait-duration-in-open-state: 5s    # 熔断器打开持续时间
        permitted-number-of-calls-in-half-open-state: 3  # 半开状态允
        automatic-transition-from-open-to-half-open-enabled: true
  retry:
    instances:
      inventory-retry:
        max-attempts: 2                    # 最多重试2次（包含首次调用）
        wait-duration: 100ms               # 重试间隔
        retry-exceptions:
          - org.springframework.web.client.HttpServerErrorException
          - java.net.SocketTimeoutException
  timelimiter:
    instances:
      inventory-tl:
        timeout-duration: 2s               # 调用超时时间
```

```java
@Service
public class OrderService {

    @Autowired
    private InventoryClient inventoryClient;

    @CircuitBreaker(name = "inventory-cb", fallbackMethod = "inventoryFallback")
    @Retry(name = "inventory-retry")
    @TimeLimiter(name = "inventory-tl")
    public String deductStock(String skuId, int quantity) {
        // 调用库存服务（可能抛异常或超时）
        return inventoryClient.deduct(skuId, quantity);
    }

    // 降级方法（熔断或重试耗尽后执行）
    public String inventoryFallback(String skuId, int quantity, Throwable t) {
        // 记录日志，返回友好提示
        return "{\"code\": 500, \"message\": \"库存服务繁忙，请稍后重试\"}";
    }
}
```

降级配置（Resilience4j）

```java
// 1. 库存服务：不可降级，熔断后直接抛出异常，订单失败
@CircuitBreaker(name = "inventoryCB")
public void deductStock(String skuId, int qty) {
    inventoryClient.deduct(skuId, qty);
}

// 2. 促销服务：可降级，返回默认优惠
@CircuitBreaker(name = "promotionCB", fallbackMethod = "getDefaultPromotion
@TimeLimiter(name = "promotionTL")
public Promotion getPromotion(String skuId) {
    return promotionClient.calculate(skuId);
}
public Promotion getDefaultPromotion(String skuId, Throwable t) {
    return new Promotion("无优惠", 0);
}

// 3. 用户积分服务：可降级，不阻塞主流程
@Retry(name = "pointsRetry", fallbackMethod = "updatePointsFallback")
public void updatePoints(String userId, int points) {
    pointsClient.addPoints(userId, points);
}
public void updatePointsFallback(String userId, int points, Throwable t) {
    log.warn("积分更新失败，用户：{}，积分：{}", userId, points);
    // 可选：写入消息队列，后续异步重试
}
```

```java
// 下单主流程，组合调用
@Transactional
public Order createOrder(OrderRequest request) {
    // 扣库存（不可降级，失败则抛异常）
    deductStock(request.getSkuId(), request.getQuantity());

    // 获取促销（降级则无优惠）
    Promotion promotion = getPromotion(request.getSkuId());
    // 更新积分（降级则仅记录日志）
    updatePoints(request.getUserId(), request.getAmount());

    // 创建订单...
    return order;
}
```

作业：服务治理

- **服务注册发现与配置**
  - 搭建Nacos环境实现服务注册、配置管理，结合Spring Cloud Gateway服务网关
  - 使用网关地址调用服务，测试动态服务路由的正确性
  - 在代码中使用Nacos的属性，测试动态更新属性能力
- **流量治理)**
  - 尝试对服务进行设置熔断、限流和降级
  - 使用JMeter进行压力测试，测试流量治理的效果